

Symmetry Reduction in the Detailed Balance Checker

The Speedup, Why It Works, and When It Can Go Wrong

1 The problem: why checking is slow

The detailed balance checker works in two phases. First it explores, by breadth-first search (BFS), all particle configurations (“states”) that the algorithm can reach from a given starting configuration. It then checks whether the algorithm satisfies *detailed balance* (DB) for every pair of states it found.

Detailed balance for a pair of states (s_i, s_j) is the condition:

$$T(s_i \rightarrow s_j) \pi(s_i) = T(s_j \rightarrow s_i) \pi(s_j).$$

Here $T(s_i \rightarrow s_j)$ is the probability of the algorithm moving from state s_i to state s_j in one step, and $\pi(s) \propto e^{-\beta E(s)}$ is the Boltzmann weight — the fraction of time a *correct* algorithm would spend in state s at equilibrium. Detailed balance is the pair-by-pair condition that guarantees the algorithm samples from the physically correct distribution. If it fails for even one pair, the algorithm is wrong.

For a connected component with N states there are $N(N-1)/2$ pairs to check. For `vmc_lattice.wl` at `NGrid = 3`, the BFS finds $N = 504$ states, giving $504 \times 503/2 = 126,756$ pairs. Most pairs have $T = 0$ in both directions (the states cannot reach each other in one step), but the symbolic computation for the $\sim 3,000$ non-trivial pairs still takes about 3 minutes on a 4-core machine. For `NGrid = 6` ($N \approx 100,000$), the pair count reaches $\sim 5 \times 10^9$ — entirely infeasible without a reduction.

The symmetry reduction proposed here cuts the number of pairs that need checking by a factor equal to the size of the algorithm’s symmetry group. This document explains the idea in plain terms, gives the quantitative impact, and then works through every condition that must hold for the reduction to be *correct* — and what happens when each one fails.

2 The core idea: symmetry creates families of equivalent pairs

What is a symmetry of the algorithm?

Consider a 3×3 square lattice with periodic boundary conditions (think of the edges as wrapping around like a torus, so there is no “edge” or special position). A *translation* by (a, b) shifts every particle one step: the particle at site (r, c) moves to $(r + a, c + b) \bmod 3$. There are nine translations in total (three choices each for a and b).

We say the algorithm is *translation-invariant* if shifting the entire configuration by (a, b) leaves all transition probabilities unchanged. Formally, writing $g \cdot s$ for the state obtained by applying translation g to every particle in state s :

$$T(g \cdot s \rightarrow g \cdot s') = T(s \rightarrow s') \quad \text{for every translation } g \text{ and every pair of states } s, s'. \quad (1)$$

In plain language: the algorithm cannot tell which corner of the lattice it is in. This holds whenever (a) the seed particle is chosen uniformly from all occupied sites (not favouring any particular position), and (b) the energy depends only on the distances between particles, not on their absolute positions. Both conditions hold for `vmmc_lattice.wl`.

The Boltzmann weight satisfies the same property: if two configurations are identical up to a translation, they have the same energy and therefore the same Boltzmann weight: $\pi(g \cdot s) = \pi(s)$.

Why this means pairs come in families

Each pair of states $\{s, s'\}$ has nine *translated copies*: $\{g \cdot s, g \cdot s'\}$ for each of the nine translations g . Because the algorithm is translation-invariant, every one of these nine pairs has *exactly the same transition probabilities* as the original:

$$T(g \cdot s \rightarrow g \cdot s') = T(s \rightarrow s'), \quad T(g \cdot s' \rightarrow g \cdot s) = T(s' \rightarrow s).$$

So the detailed-balance expression is the same for all nine pairs. If it equals zero for one of them, it equals zero for all nine.

The key argument: checking one pair certifies nine

Suppose detailed balance holds for the original pair (s, s') :

$$T(s \rightarrow s') \pi(s) = T(s' \rightarrow s) \pi(s').$$

Now consider any translated copy $(g \cdot s, g \cdot s')$. We check each side of the detailed-balance condition:

$$\begin{aligned} T(g \cdot s \rightarrow g \cdot s') \pi(g \cdot s) &= T(s \rightarrow s') \pi(s) && \text{[translation-invariance of } T \text{ and } \pi] \\ &= T(s' \rightarrow s) \pi(s') && \text{[detailed balance for the original pair]} \\ &= T(g \cdot s' \rightarrow g \cdot s) \pi(g \cdot s'). && \text{[translation-invariance again]} \end{aligned}$$

The first and last expressions are exactly the two sides of detailed balance for the translated pair. They are equal, so detailed balance holds for the translated pair too — *without us having to check it*.

The reverse works identically: if detailed balance *fails* for the original pair, it fails for every translated copy as well. So checking one pair from each family of nine is both necessary and sufficient.

Rotational and reflective symmetry: D4

The 3×3 square lattice has additional symmetry: rotating all particle positions by 90° , 180° , or 270° , or reflecting across the horizontal, vertical, or either diagonal axis, leaves the physics unchanged. There are 8 such operations (4 rotations including the identity, and 4 reflections), together called the D4 symmetry group of the square.

Combined with the 9 translations, there are $8 \times 9 = 72$ operations in total. If the algorithm is invariant under all 72 of these, each family grows to 72 equivalent pairs, and the number of pairs to check shrinks by a factor of 72.

For `vmmc_lattice.wl` this D4 invariance does hold: the displacement list $\{(\pm 1, 0), (0, \pm 1), (\pm 1, \pm 1)\}$ is unchanged by any rotation or reflection of the square, and the energy (depending only on inter-particle distances, which rotations and reflections preserve) is also invariant.

3 Quantitative impact

Quantity	No symmetry	Translations ($ G = 9$)	D4+trans. ($ G = 72$)
States in component	504	504	504
Pairs to check	126,756	14,084	$\approx 1,760$
Unique sym. exprs	$\sim 3,000$	~ 333	~ 42
Estimated runtime	3 min	~ 20 s	$\sim 2\text{--}3$ s

For `NGrid` = 6 ($N \approx 100,000$ states), the pair count without symmetry is $\sim 5 \times 10^9$ — computationally infeasible. With translations ($|G| = 36$) this reduces to $\sim 1.4 \times 10^8$, and with D4 + translations ($|G| = 288$) to $\sim 1.7 \times 10^7$. At this scale the symmetry reduction is not an optimisation; it is the difference between a feasible and an infeasible computation.

4 Concern: what about pairs of states that are related by symmetry?

A natural worry: two states can sometimes be *related by a symmetry* — for example, a particle cluster translating by one site produces a state that is a translated copy of the original. These two states are in the same translation family. The transition between them is a pair whose two members are in the same family. Do we not need to check such pairs independently?

The answer is no — these pairs are already counted in the family structure and are not a special exception.

Let s be a state and $h \cdot s$ be a translated copy of it (for some non-trivial translation h). The pair $\{s, h \cdot s\}$ is a perfectly ordinary unordered pair, with its own family of nine translated copies. Detailed balance for this pair reads (using $\pi(s) = \pi(h \cdot s)$ since the states have the same energy):

$$T(s \rightarrow h \cdot s) = T(h \cdot s \rightarrow s).$$

This is a *proposal-symmetry* condition: the probability of the cluster moving by displacement h must equal the probability of the reverse displacement h^{-1} . For `vmmc_lattice.wl` the displacement list always contains $\mathbf{d}$ and $-\mathbf{d}$ as a pair, so this holds automatically and these pairs need no explicit checking.

Even without that argument: the number of “within-family” pairs like this grows only linearly with the number of states ($O(N)$), whereas “between-family” pairs grow quadratically ($O(N^2)$). For large N the within-family contribution is negligible, and the dominant $O(N^2)$ cost still shrinks by the full symmetry factor.

5 Conditions for correctness: what can go wrong

The argument above is logically sound, but it rests on several conditions that must be *verified*, not assumed. This section lists each condition, explains why it might fail, and assesses the consequence of getting it wrong. A failure in any one of these conditions can turn a correct FAIL into an incorrect PASS — the most dangerous kind of error for a checker.

5.1 Condition 1: The algorithm must actually be symmetric

The entire reduction rests on equation (1): that the algorithm’s transition probabilities are genuinely unchanged when the whole configuration is translated (or rotated or reflected). If this

fails for even one pair of states and one symmetry operation, the reduction can certify a violating pair as passing.

A concrete failure: the biased algorithm. The file `vmmc.biased.wl` provides an exact demonstration. That algorithm proposes rightward displacements twice as often as leftward ones, breaking proposal symmetry. Now consider a pair of states (s, s') where s' is s with a particle cluster shifted one site to the right. This pair violates detailed balance because the forward rate (rightward, doubled probability) differs from the reverse rate (leftward, normal probability).

A D4 symmetry reduction would group this pair into a family of 72 equivalent pairs. Among those 72 is a pair related by a 90° rotation, where the cluster moves upward instead of rightward. Upward and downward moves are *not* biased, so that pair satisfies detailed balance perfectly. If the checker happens to pick the upward pair as the family representative, it evaluates it, finds no violation, and certifies the whole family as passing — including the rightward pair that actually fails. The checker reports PASS on a broken algorithm.

Why verification is harder than it looks. The natural verification strategy is: run the BFS from state s , run it again from the rotated state $g \cdot s$, and check whether the two sets of transition probabilities agree (after rotating the second set back by g^{-1}).

The difficulty is that the two BFS runs produce symbolic expressions in different forms. The BFS threads random-choice variables (labelled $r_1, r_2, \dots$) through the algorithm in the order it encounters them, and two different starting states encounter the same decisions in different orders. The resulting symbolic expressions can be mathematically equal but written completely differently. A character-by-character string comparison would declare them unequal when they are in fact equal — a false alarm. A proper check requires simplifying the difference to zero using a symbolic algebra system, which can be as expensive as the DB check itself.

An alternative is to verify by reading the algorithm’s source code: confirm that the seed-particle selection, displacement probabilities, and link-weight computation each depend only on inter-particle geometry (not absolute positions), so that rotating or translating the whole configuration provably leaves everything unchanged. This is more reliable but requires human review of every algorithm before applying the reduction.

5.2 Condition 2: The connected component must be closed under the symmetry

The BFS finds a connected component: a set of states all reachable from one another via the algorithm. For the family reduction to work within one component, every symmetry image of every state in that component must also be in the *same* component.

If this fails — if translating state s (which is in component C) gives a state $g \cdot s$ that is in a *different* component C' — then the family of the pair $\{s, s'\}$ (both in C) contains pairs $\{g \cdot s, g \cdot s'\}$ that span C and C' . The BFS for component C never found $g \cdot s$, so those cross-component pairs were never examined. An implementation that incorrectly counts cross-component images as “certified” would be skipping checks it has not actually performed.

In practice, the safe implementation must confirm: for each state s found in a component, every symmetry image $g \cdot s$ is also in the same component. If some images fall outside, the effective symmetry group for that component is only the *sub-collection* of operations that do map the component to itself, and the speedup factor is accordingly smaller.

For `vmmc_lattice.wl` at `NGrid=3`, the 504-state component has size $504 = 56 \times 9$, consistent with all 9 translations acting within it — but this must be confirmed explicitly, not assumed from the arithmetic.

5.3 Condition 3: BFS must run on all states, not just “representative” states

A tempting shortcut is: pick one representative state per symmetry family, run BFS only from representative states, and then check only pairs of representatives. This is **wrong** and produces incorrect transition probabilities.

Here is the problem. Say state s is mapped to its representative by applying symmetry operation g_0 : $\text{representative}(s) = g_0 \cdot s$. State s' is mapped to its representative by a *different* operation g_1 : $\text{representative}(s') = g_1 \cdot s'$. If we run BFS only from representatives, the transition we compute is $T(g_0 \cdot s \rightarrow g_1 \cdot s')$ — the probability of moving from the representative of s to the representative of s' in a single step.

But by the symmetry property, $T(g \cdot s \rightarrow g \cdot s') = T(s \rightarrow s')$ only when the *same* operation g is applied to both states. With $g_0 \neq g_1$, the quantity $T(g_0 \cdot s \rightarrow g_1 \cdot s')$ is a transition between two states that are not symmetry-images of each other in the required sense, and is generically *not* equal to $T(s \rightarrow s')$. The BFS-on-representatives shortcut computes the wrong numbers.

The correct implementation always runs the full BFS on all N states, computing the actual transition probabilities between actual states. The symmetry reduction only controls *which pairs are subsequently sent to the DB checker* — it does not change how transition probabilities are computed.

5.4 Condition 4: Do not assume all families have the same size

The speedup formula $N(N-1)/(2|G|)$ assumes every family of pairs has exactly $|G|$ members. This fails when some symmetry operation maps a pair to itself.

States that look the same after a symmetry. If a symmetry operation g (other than the identity) maps a state s to itself — $g \cdot s = s$ — then the family of s has fewer than $|G|$ members. As a concrete example: on a fully-occupied 3×3 lattice where every site holds a particle of the same type, any translation maps the occupancy to itself, so all 9 translations fix the state.

If an implementation *assumes* every family has exactly $|G|$ members and after checking one representative automatically skips the next $|G| - 1$ pairs, it will sometimes skip pairs from *different*, smaller families — potentially missing a violation. The safe approach is to compute the canonical representative for every pair explicitly, and only check a pair if it *is* its own canonical representative, without ever assuming how large the family is.

Symmetries that swap the two states in a pair. If $g \cdot s = s'$ and $g \cdot s' = s$ (a symmetry swaps the two states in the pair), then by symmetry invariance $T(s \rightarrow s') = T(s' \rightarrow s)$, and since $\pi(s) = \pi(s')$ (same energy), detailed balance holds automatically. These pairs can be identified and skipped entirely with no symbolic computation needed.

5.5 Condition 5: Rotations must be correctly implemented for the lattice size

For a 90° rotation on an $L \times L$ lattice, the rotation must map every lattice site to another lattice site — a bijection (a one-to-one mapping with no overlaps or gaps). The standard formula rotates site (r, c) to $(c, L - 1 - r)$. For $L = 3$: $(0, 0) \rightarrow (0, 2)$, $(1, 0) \rightarrow (0, 1)$, $(2, 0) \rightarrow (0, 0)$, and so on — all 9 sites map to the 9 sites with no repeats. For $L = 4$: $(0, 0) \rightarrow (0, 3)$, $(3, 0) \rightarrow (0, 0)$, etc. — again a valid bijection.

An off-by-one error in this formula (for example, using L instead of $L - 1$ in the formula) would produce a mapping where two different sites map to the same image, or where some images fall outside the grid. The result would be a rotation that is not actually a symmetry of the lattice, so the “family” structure would be wrong. Pairs that the implementation considers equivalent

would not be genuinely equivalent, and pairs it skips might have been in different families that were never checked. This kind of bug is silent: the code produces a result without error, but that result is wrong.

5.6 Condition 6: The speedup partially overlaps with the existing symbolic cache

The current checker already deduplicates symbolic expressions: if two pairs (s_i, s_j) and (s_k, s_l) produce *identical* symbolic expressions for $T(s_i \rightarrow s_j)\pi(s_i) - T(s_j \rightarrow s_i)\pi(s_j)$, the FullSimplify call is done only once and the result reused. For a genuinely symmetric algorithm, all 9 pairs in the same translation family produce the same symbolic expression (because the coupling-constant atoms depend only on particle type and distance, both of which are preserved by translations). The existing cache therefore already achieves the $\sim 9\times$ reduction in the number of FullSimplify calls.

This means the symmetry orbit reduction, as proposed, *does not* speed up the unique-expression evaluation step — that is already cached. Its real gain is in reducing the number of *pairs that are constructed and pre-processed* before reaching the symbolic evaluator. For `NGrid` = 3 ($N = 504$), constructing 126,756 pairs and filtering them down to $\sim 3,000$ non-trivial ones is fast; the bottleneck is FullSimplify, and the cache already helps there. For `NGrid` = 6 ($N \approx 100,000$), constructing 5×10^9 pairs is itself the bottleneck, and the symmetry reduction from 5×10^9 to 1.7×10^7 is what makes the computation feasible at all. The speedup therefore matters most at the scales where it is most needed.

5.7 Condition 7: “It works on my test cases” is not sufficient validation

For the “90° clockwise rotation only” algorithm described separately, the D4 symmetry checker happens to produce the correct FAIL result *even though the algorithm is not D4-invariant*. The reason is that reflecting a “clockwise-violated” pair gives a “counter-clockwise-violated” pair, which is also a violation, just with the two sides swapped. Every member of every D4 family is a violating pair, so whichever representative the checker picks, it finds a violation.

This *accidental correctness* is dangerous as a validation strategy: if you test the symmetry checker only on that broken algorithm and observe correct FAIL results, you might conclude the G-invariance verification is working when it is not. A proper test requires an algorithm like `vmmc_biased.wl`, which is broken in a way that creates *non-violating* members in the same family as violating ones. Without such targeted tests, a buggy or absent G-invariance verification could go undetected.

The general principle: use D4 symmetry only when D4 invariance has been verified for the specific algorithm. An algorithm that is translation-invariant but not D4-invariant should use only the translation group for the reduction. Using a symmetry group *larger* than the algorithm’s actual symmetry is not a harmless over-reach; it actively introduces false negatives.

6 What a correct implementation requires

Realising the speedup safely requires four steps.

1. **Verify the algorithm’s symmetry before applying any reduction.** For each symmetry operation g in the chosen group, check that BFS from state s and BFS from $g \cdot s$ produce equivalent transition structures. This comparison must use symbolic simplification (not string matching), because the two BFS runs produce the same mathematical expressions written in different forms. Alternatively, verify by source-code inspection that the seed selection, displacement probabilities, and link-weight computation are all position-independent.

Without this step the reduction is unsound.

2. **Run the full BFS on all N states.** Do not attempt to shortcut by running BFS only on representative states. This produces incorrect transition probabilities (see Condition 3). The reduction controls which pairs are checked, not how probabilities are computed.
3. **Confirm the component is closed under the symmetry.** For each state s in the component and each symmetry operation g , verify that $g \cdot s$ is also in the same component. If some images fall outside, restrict the symmetry group to those operations that do map the component to itself, and use that smaller group's size as the speedup factor.
4. **Compute canonical representatives per pair, without assuming family size.** For each pair (s_i, s_j) , apply all symmetry operations and take the lexicographically smallest resulting pair as the canonical representative. Check the DB condition only for pairs that are their own canonical representative. Never skip pairs by assuming the family has a particular size.

None of these steps changes the *algebraic* content of the DB check. They reduce the *number of times* it is performed — but only after establishing that this reduction is logically sound for the specific algorithm and connected component being checked.

7 Conclusion

The symmetry reduction is mathematically sound: if an algorithm and its energy function are both invariant under a symmetry group G , then checking detailed balance for one representative per family of $|G|$ equivalent pairs is logically equivalent to checking all pairs. The reduction is exact, not approximate.

However, the reduction is **safe to use only if all of the following conditions are satisfied**:

- The algorithm's symmetry is verified symbolically — not assumed from the physics or from passing a few test cases.
- The connected component found by BFS is closed under the symmetry group (every symmetry image of every state in the component is also in the component).
- The full BFS is run on all N states; BFS is not short-circuited using canonical states.
- Canonical representatives are computed per pair without assuming family size.
- Rotation operations are correctly implemented for the specific lattice size, with no off-by-one errors.

A failure in the first condition in particular can silently convert a correct FAIL into an incorrect PASS. The `vmmc.biased.w1` example demonstrates this concretely: an algorithm with a rightward bias fools a D4 symmetry checker because rotating the bias by 90° produces an unbiased upward move, and the unbiased pair becomes the family representative.

The speedup is most valuable at $\text{NGrid} \geq 5$, where the pair count without symmetry is computationally infeasible. At $\text{NGrid} = 3$ the existing symbolic deduplication cache already captures most of the same benefit in the bottleneck step (FullSimplify calls).